Efficient Path Query Processing in Relational Database Systems

Anonymous Author(s)

Abstract

Path queries are crucial for property graphs, and there is growing interest in supporting not only regular expressions over labels, but also constraints on the property values of vertices and edges. Efficient evaluation of such queries requires early filtering of intermediate results, but state-of-the-art (SOA) graph and relational DBMS struggle to effectively exploit property constraints. We introduce the notion of negatively stable constraints and propose ReCAP, a novel abstraction that leverages it for holistically specifying early filtering opportunities in path queries. To create a ReCAP it suffices to (1) specify the regular expression for the label constraint (as in the MATCH clause of Cypher or GQL), (2) define functions to manage state (like for a user-defined aggregate), and (3) define a Boolean function to identify early filtering opportunities on the state. We design general constructions for common constraint types, together with a default ReCAP for query conditions that are not amenable to early filtering—in which case our technique behaves like SOA graph and relational DBMS. We also propose a recursive SQL query template to execute any ReCAP instance in a standard relational DBMS. When early filtering opportunities on property constraints exist, our implementation in DuckDB achieves speedups up to 400,000× over the competitors.

CCS Concepts

• **Information systems** → **Query optimization**; *Structured Query Language*.

Keywords

path query, property graph, recursive SQL query abstraction

ACM Reference Format:

Anonymous Author(s). 2018. Efficient Path Query Processing in Relational Database Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Many problems in diverse domains can be modeled as graphs, where vertices represent entities and edges the relationships between them. Examples include social, biological, and financial transaction networks. Analyzing such data often involves querying for paths that satisfy certain constraints [2] including: regular expressions over edge labels, monotonicity constraints on timestamps (e.g., to ensure causality), aggregate constraints on weights (e.g., to ensure quality), or combinations thereof. While path queries have been studied for decades, producing many theoretical insights [35], integrating those techniques into practical data management systems is "still in its experimental stage" [45]. Recent graph query language standards like GQL [18] and SQL/PGQ [1] enable users to design powerful declarative path queries, hence there is an urgent need for techniques to efficiently execute such queries.

We focus on path-query processing in *relational DBMS* for two reasons. First, they are ubiquitous in practice. If a system that is already up and running can efficiently support the target queries, this avoids the additional complexity and cost of managing specialized engines for each type of problem (and moving data between them). Second, as Pavlo and Stonebraker [52] point out, relational and graph DBMS will likely converge to a single engine that supports both relational and graph data management. That should certainly apply to path queries, because they are equivalent to join-heavy relational queries with selections and projections [26]. Hence our results also transfer in principle to graph DBMS.

Example 1 (Query Q_A). *Consider a property graph of financial transactions, where vertices represent accounts and edges the transactions between them (Figure 1). A user wants to find all paths up to length ℓ starting from account s, such that (1) transactions along the path are in chronological order (strictly increasing timestamps), (2) no edge repeats (trail semantics), and (3) the transferred amounts are relatively "homogeneous", meaning that the difference between maximum and minimum transaction amount along the path is upper bounded by some parameter U.*

Executing such queries efficiently is far from trivial as Figure 2 indicates. We report running times of Q_A on a Bitcoin [30] transaction graph with 6k vertices and 36k edges. Despite the small input, SOA relational (DuckDB [50]) and graph DBMS (Neo4j [46], Kùzu [15]) fail to scale with path length.[1] A look at the query plans reveals that they suffer from huge intermediate results because they do not push the conditions down effectively. In contrast, our ReCAP abstraction (Section 4) enabled DuckDB to scale to more than double the path length.

Why did the optimizers of DuckDB, Neo4j, and Kùzu fail to push the conditions down effectively? Let us take a closer look at DuckDB. (The situation is analogous for the graph DBMS.) Listing 1 shows Q_A in SQL. It uses a recursive Common Table Expression (CTE) to define all paths up to length ℓ, accumulating the amounts and timestamps along the path in arrays. The outer WHERE clause filters paths based on the max-min constraint and the timestamp ordering condition. Cypher-based graph DBMS would similarly rely on list processing over the paths identified in the MATCH clause. The reason for the poor scalability does *not* lie in the ordering of the clauses: SQL and Cypher are declarative languages, and the order of clauses does not imply evaluation order.

The core challenge lies in the complex interplay of recursive joins, inequality conditions, and aggregate constraints.

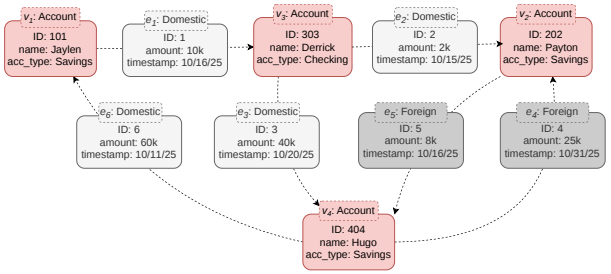
[1] Experiments with other SOA DBMS yielded similar results and are omitted for brevity.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX



Accounts (Vertices)

ID	name	acc_type
101	Jaylen	Savings
202	Payton	Savings
303	Derrick	Checking
404	Hugo	Savings

Transactions (Edges)

ID	src	dst	amount	time	label
1	101	303	10k	10/14/25	Domestic
2	303	202	2k	10/15/25	Domestic
3	303	404	40k	10/20/25	Domestic
4	402	202	25k	10/31/25	Foreign
5	202	404	8k	10/16/25	Foreign
6	404	101	60k	10/11/25	Domestic

Figure 1: On the left: Graph representing accounts (vertices) transactions between them (edges). The label is shown next to the ID; properties are shown inside the box. On the right: Tabular representation of graph.

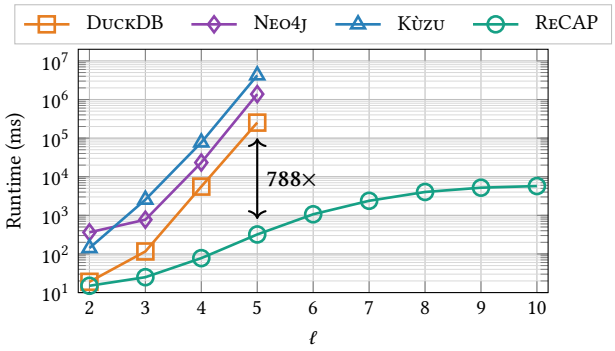


Figure 2: Runtime of RECAP vs. SOA competitors for varying path-length limits. All competitors scale poorly and exceed a 2-hour timeout for $\ell > 5$.

Listing 1: SQL query for Example 1

```
WITH RECURSIVE Paths AS (
    SELECT s AS v,
           0 AS path_length,
           [] AS amounts,
           [] AS times,
           [] AS edge_ids
    UNION ALL
    SELECT E.dst AS v,
           P.path_length + 1,
           P.amounts + [E.amount] AS amounts,
           P.times + [E.time] AS times,
           P.edge_ids + [E.id] AS edge_ids
    FROM Paths P
    JOIN Edges E ON P.v = E.src
    WHERE p.path_length < ℓ
)
SELECT *
FROM Paths
WHERE list_max(amounts) - list_min(amounts) <= U
AND len(edge_ids) = len(list_distinct(edge_ids))
AND NOT list_contains([times[i] <= times[i-1]
    FOR i IN range(2, len(times)+1)], true)
```

Consider the seemingly simple timestamp monotonicity, which could be checked any time an edge is appended to a path. For example, for a 3-way self-join of the edge relation, a condition like $e_1.\texttt{time} < e_2.\texttt{time} \wedge e_2.\texttt{time} < e_3.\texttt{time}$ would be pushed down to the corresponding binary joins. Then why can the query optimizer not figure this out for Listing 1? Notice how the time constraint is encoded: instead of a 3-way join where the number of relations is fixed and each of them can be referenced explicitly, the result path could have any length. Hence one must express the condition in a way that applies to any path length. The resulting `list_contains(...)` statement is opaque to the optimizer, suggesting that all edge values are needed to evaluate it. This forces the system to check it in the very end on complete path candidates, instead of during the join that assembles these paths. The same applies to the enforcement of trail semantics via the list of edge IDs. For the max-min difference on the transaction amounts, the situation is even more challenging, because computing a path aggregate like the maximum across all edges indeed requires knowing all edges. Interestingly, our approach can still apply effective early filtering to it.

To address this problem in a principled manner, we introduce the notion of a *negatively stable* constraint: if it does not hold on a path p, then no matter what edges are appended to p, it can never hold on any longer path with prefix p. All three constraints in Q_A are negatively stable. E.g., once the timestamp monotonicity is violated for any pair of adjacent edges, the path is *doomed*: adding more edges cannot repair it. A system that recognizes negatively stable constraints can discard doomed prefixes the moment they are detected, rather than extending them to full length and discarding them in the end. This structural property is not limited to simple predicates: it extends naturally to aggregate constraints, regex-dependent thresholds, and combinations of label and property conditions. We show how to express this structure precisely and how to exploit it effectively within a relational engine:

(1) We identify *negative stability* as a crucial structural property: if a constraint does not hold on a path, then no extension of that path can satisfy it. Our novel RECAP (**Re**gular expressions, **C**onstraints, and **A**ggregation **P**atterns) abstraction exposes negative stability so that a standard relational DBMS can apply early filtering during path exploration. RECAP is fully general: it supports all combinations of regular expressions over edge labels and data constraints on edge and vertex properties. When constraints do not admit early filtering, RECAP reduces to the query plans used by current SOA systems that apply "difficult" property constraints at the end.

(2) Implementing a ReCAP is similar to specifying a user-defined aggregate (UDA) on property values, requiring 5 functions to initialize, maintain, and finalize state, and to determine if a path can be filtered early. Through a `CASE` statement, these functions can modify their behavior based on the state of the label pattern, thus enabling early filtering based on complex constraints that involve dependencies between label pattern and property values.

(3) We demonstrate that ReCAP can be implemented with little coding effort in SQL, using a standard recursive select-project-join (SPJ) query. We also propose effective optimizations including automaton-style processing converted to efficient joins, dictionary flattening, and function inlining that leverage the full power of SQL. Our ReCAP implementation in DuckDB achieves orders of magnitude speedup and scales to significantly longer paths compared to SOA graph and relational DBMS.

2 Problem Definition

We study path queries over property graphs. For simplicity, our discussion *ignores vertex labels and properties*, focusing on edges. Extending our approach to vertices is straightforward.

Definition 1 (Property Graph [2]). *A* property graph G *is a 5-tuple* $(V, E, \rho, \lambda, \sigma)$ *where* V *is a finite set of vertices,* E *is a finite set of edges with* $V \cap E = \emptyset$, $\rho : E \to (V \times V)$ *is a total function that maps an edge to its source and target vertex,* $\lambda : (V \cup E) \to L$ *is a total function that assigns a label from label set* L *to each vertex and edge, and* $\sigma : (V \cup E) \times \text{Prop} \to \text{Val}$ *is a partial function that assigns properties to vertices and edges. Each property is a key-value pair with a key from* Prop *and a value from* Val. *In addition to scalars, we also allow complex values such as lists and dictionaries.*

Example 2. *In Figure 1,* $\rho(e_2) = (v_3, v_2)$, $\lambda(e_4) = \texttt{Foreign}$, $\lambda(n_1) = \texttt{Account}$, *and* $\sigma(e_5, \texttt{amount}) = 8\text{k}$.

Definition 2 (path, prefix, extension, word). *A* path p *of length* j *is a sequence of vertices and edges* $v_0 e_1 v_1 \cdots e_j v_j$ *for* $j \geq 1$, *where* $e_i \in E$ *and* $\rho(e_i) = (v_{i-1}, v_i)$ *for each* $1 \leq i \leq j$. *Any* $p' = v_0 e_1 v_1 \cdots e_i v_i, i \leq j$ *is a* prefix *of* p; *and* p *is an* extension *of* p'. *We write* $\Pi(G)$ *for the set of all paths in* G. $\Pi_\ell(G)$ *denotes the set of all paths in* G *of length at most* ℓ. *The word* $w(p)$ *of a path* p *is the concatenation of the labels of each edge in* p.

Example 3. *In Figure 1 the path* $p = v_3 e_2 v_2 e_5 v_4$ *from Derrick's account to Hugo's account, generates the word* $\texttt{Domestic} \cdot \texttt{Foreign}$.

Definition 3 (regular expression, language). *A* regular expression R *over a label set* L *is defined in the standard way [51]. The* language recognized by R, *written* $\mathcal{L}(R)$, *is the set of strings over* L *generated by* R.

Definition 4 (path query). *A* path query $Q = (S, R, \varphi)$ *over a property graph* G *is defined by a set of start vertices* $S \subseteq V$, *a regex* R *over label set* L, *and a Boolean function* φ *defined for all paths over* G. *It returns a set of paths over* G *such that each result path* $p = v_0 e_1 v_1 \cdots e_j v_j$ *satisfies the following properties:*

- $v_0 \in S$,
- $w(p) \in \mathcal{L}(R)$, *and*
- $\varphi(p) = \texttt{true}$.

A length-limited path query Q_ℓ *further requires* $j \leq \ell$.

We support other variations of the path query: no start vertex given, start vertex and end vertex given, or that the path must cross through some specific vertices "in the middle". However, we will focus on path queries that originate from a given start vertex s. Our technique relies on classic finite automata (NFA) to process regular expressions:

Definition 5 (NFA). *The NFA for path query* $Q = (S, R, \varphi)$ *over property graph* G *is a 5-tuple* $N = (\mathfrak{Q}, L, \delta, q_0, \mathfrak{Q}_F)$, *consisting of*

- *A set of states* $\mathfrak{Q}$, *a single initial state* $q_0 \in \mathfrak{Q}$, *and a set* $\mathfrak{Q}_F \subseteq \mathfrak{Q}$ *of accepting states.*
- *A label set* L *(the same as the property graph's label set).*
- *A transition function* $\delta : \mathfrak{Q} \times L \to \mathcal{P}(\mathfrak{Q})$, *where* $\mathcal{P}(\mathfrak{Q})$ *denotes the power set of* $\mathfrak{Q}$.

The language recognized by N *is equivalent to the language specified by* R, *i.e.,* $\mathcal{L}(N) = \mathcal{L}(R)$.

Note that Definition 4 includes cases that are well supported by SOA graph DBMS, such as path queries with (selective) label regex only, or those where property constraints can be attached to individual edges within the regex pattern. For example, Cypher's `MATCH (s) -[e1]-> (v1) -[e2 WHERE e1.time < e2.time]-> (t)` enforces timestamp monotonicity between 2 named edges of a 2-hop path. Since SOA graph DBMS can already efficiently compute such queries,[2] we focus on "difficult" instances with iterating patterns where path length is not pre-determined. Query Q_A's regex pattern corresponds to `MATCH ()-[]->{1,}()`, therefore one cannot express timestamp monotonicity on its edges there [20], which forces Cypher implementations to resort to list processing [21]. To enable efficient processing of such queries, one must identify structural properties that enable early filtering of intermediate results that cannot lead to a query answer:

Definition 6 (Doomed and viable intermediate paths). *Let* $Q = (S, R, \varphi)$ *be a path query on property graph* G. *Path* $p' \in \Pi(G)$ *is* doomed *if none of its possible extensions can be a query answer; otherwise it is* viable.

Whether an intermediate path is doomed depends on both label constraint R and property constraint φ. The former is determined by the NFA: if the current state has no path to an accepting state, the intermediate result is doomed. For the latter, the analogue is a constraint guaranteeing that once it fails on a path p', it must fail on every extension of p':

Definition 7 (Negatively stable constraint). *Let* φ *be a Boolean function as defined in Definition 4. We say* φ *is* negatively stable *if for every path* $p' \in \Pi(G)$ *and every extension* p *of* p'

$$\varphi(p') = \texttt{false} \implies \varphi(p) = \texttt{false}.$$

Lemma 1. $\varphi = \bigwedge_i \varphi_i$ *is negatively stable, if any* φ_i *is negatively stable.*

Example 4 (Negatively Stable Constraints in Q_A). *All property constraints from* Q_A *are negatively stable:*

[2] Leading graph DBMS *do not support all regular expressions*, but our technique does.

(i) Monotonicity of timestamps*: consecutive-edge timestamp conditions. Once violated by adjacent edges e_i, e_{i+1}, no suffix can retroactively repair the violation.*

(ii) Trail semantics*: no repeated edge. Once an edge is repeated, every extension still contains the repetition.*

(iii) Upper bound on aggregate range*: $\max - \min \leq U$ over edge amounts. The gap $\max - \min$ is monotonically non-decreasing under path extension, so once it exceeds U it remains above U.*

When φ is negatively stable, the moment $\varphi(p') = \texttt{false}$ on an intermediate path p', p' is doomed and can be discarded.

3 Running Example

We introduce an example query that highlights all features of ReCAP and motivates the general abstraction.

Example 5 (query Q_B). *In the financial-transaction graph from Example 1, find all paths from a given* `Account` *s, such that:*

(1) The labels match pattern `Domestic`$^+$`Foreign`*, i.e., at least one* `Domestic` *transaction followed by exactly one* `Foreign` *transaction.*

(2) Consecutive `Domestic` *transactions are executed within 2 days of each other, i.e., their timestamp difference falls in range $[-2, 2]$ days.*

(3) The last `Domestic` *and the* `Foreign` *transaction are executed within 3 days of each other, i.e., their timestamp difference falls in range $[-3, 3]$ days.*

(4) No edge is repeated (trail semantics).

For each result path, return the sequence of its edge IDs.

Query Q_B generalizes Q_A by introducing a label regex, creating a dependency between label pattern and property constraint (different timestamp intervals), and generalizing the timestamp monotonicity. The latter allows searching for suspicious transfers that attempt to hide the money flow by inverting transaction order: To hide that \$10 move from A to B to C, B could first send \$10 to C, then soon after get "reimbursed" by A.

It is easy to identify opportunities for early filtering when considering the next transaction edge to be appended to a path:

- Use an NFA to efficiently check the label pattern.
- Maintain `last_time`, the time of the path's last transaction. Filter out transaction e if e.`time` − `last_time` falls outside the time range (2 or 3 days, respectively).
- Maintain `edge_ids`, a list of IDs of all edges along the path. Filter out transaction e if e.`id` ∈ `edge_ids`.

The challenges here are:

- How to efficiently implement an NFA in a relational engine?
- How to implement early filters that depend on both the state of the label pattern and the data values (e.g., time range 2 vs 3)? In the example, it suffices to check e.`label`, but in general it depends on the state transition of the NFA, which the relational engine must keep track of.
- How to implement state that grows with path length, like the list of edge IDs? In a recursive SQL query the schema is fixed, hence state must fit into a fixed set of columns.



Transitions T

from_state	to_state	label
1	2	Domestic
2	2	Domestic
2	3	Foreign

Figure 3: Left: NFA for regex `Domestic`$^+$`Foreign`. Right: Tabular representation of the NFA.

Listing 2: NFA in SQL

```
WITH RECURSIVE Paths AS (
    SELECT s as v,
           q0 as q
    UNION ALL
    SELECT E.dst as v,
           T.to_state as q,
    FROM Paths P
    JOIN Edges E ON P.v = E.src
    JOIN Transitions T ON T.from_state = P.q
    WHERE T.label = E.label
)
SELECT ... FROM Paths WHERE q IN Ω_F
```

4 The ReCAP Abstraction

We propose ReCAP to address the above challenges. It streamlines the process of defining early filter opportunities through an automaton model that augments an NFA with a dictionary that maintains key-value pairs. In contrast to previous work on similar automata with memory, our goal is **not** to navigate the tradeoff between expressiveness and complexity, but to **simplify the specification of early filtering opportunities and enable syntactic conversion of the specification to an executable SQL query that applies those early filters**. We will present a general construction for any path query $Q = (S, R, \varphi)$ and illustrate it for Q_B. For simplicity, we set $S = \{s\}$, i.e., the query has a single start vertex s. It is straightforward to generalize our construction to any set $S \subseteq V$.

4.1 Regex as a Relation

NFAs are widely recognized as an efficient means of evaluating regular expressions, from data streams [7, 8, 23] to label patterns on paths [5, 13, 19]. Hence, the first step in constructing the ReCAP for a path query $Q = (S, R, \varphi)$ is to use standard textbook techniques to generate the NFA for Q's label regex R. Figure 3 shows the NFA for Q_B, together with its tabular representation T. Each row in the table corresponds to a tuple `(from_state, to_state, label)`.

The input to this NFA is *not* a sequence of tokens (like for regular languages), but *conceptually* it is the set of all paths in G that start from a vertex in S. This is equivalent to traversing the product-graph of the NFA and input graph G [5, 43], for which SOA constructions rely on custom implementations. We note that the product-graph is equivalent to a join between NFA and G, when representing the former as as a transition table (see Figure 3) and the latter as a table of edge tuples. We therefore propose to implement it in SQL as a recursive join with a Common Table Expression (CTE) as shown in Listing 2, avoiding custom implementations and instead relying

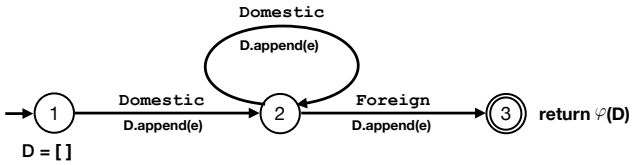


Figure 4: Default construction for property-constraint evaluation. When transitioning upon appending a new edge e to a path, all relevant edge data is stored in list D. In an accepting state, property constraint φ is evaluated on D.

on the full power of relational engines, which are designed and optimized for efficient join execution.

The CTE's anchor member initializes the start vertex of the graph and the start state of the NFA. The recursive member joins intermediate paths with the edge set, thus expanding them by one more edge. Notice how the WHERE clause enforces not only valid connections between graph edges, but also that a valid NFA transition must be taken. Non-determinism occurs when an intermediate path can make multiple transitions with the same edge, i.e., multiple transition tuples join with this path and edge. It is easy to show that this query implements the classic product construction [43], but now leveraging the power of relational engines. The NFA is a table, the join condition `T.label = E.label` is a standard equi-join predicate the optimizer can reason about, and invalid label sequences are eliminated early during each join step. Our approach supports *any regex* over labels, generalizing those supported by Cypher's `MATCH` clause. The query that calls the CTE enforces that only paths whose label sequence reaches an accepting state will be returned. The dots indicate that one could return any information about the matched path, as long as that information was collected by the recursive query.

4.2 Property Constraints: Default Construction

We now extend Listing 2 to also support property constraint φ, first presenting a default construction. The next section will extend it for early filtering on properties.

Notice that the output of the j-th iteration of the recursive query in Listing 2 is the set of intermediate j-hop paths. The next iteration joins them with the transitions and graph edges, producing all intermediate paths of length $j+1$, and so on. In Listing 2 only the last vertex v and the NFA state q are stored. To support property constraints, the corresponding property values must be collected, too.[3] We implement this by extending Listing 2 as follows:

- Add a new column D to the schema of the Paths relation. Initialize it as the empty list in the anchor member.
- In the recursive member (when joining a path tuple p with an edge tuple e and an NFA transition tuple t) append all relevant property data from e to $p.D$.
- In the outer query's WHERE clause, check if $\varphi(D) = \texttt{true}$.

This default construction is illustrated for Q_B in Figure 4. It is equivalent to the execution plan used for "difficult" constraints by SOA graph DBMS such as Neo4j, Kùzu, and Memgraph. The checking of $\varphi(D)$ corresponds to list processing in Cypher [21]. Applying list processing after label-based matching wastes computation on intermediate results that are *doomed*, i.e., can never become a query result:

Example 6 (Wasted Computation). *Consider Q_B on the graph in Figure 1, starting at Derrick's account. Path $p = v_3e_3v_4e_6v_1e_1v_3e_2v_2e_5v_4$ (from Derrick to Hugo) matches the label regex, but fails property constraint φ. Since its prefix $p' = v_3e_3v_4e_6v_1$ already fails the timestamp-range constraint, meaning p' is doomed (Definition 6), no extension of p' can ever satisfy φ and the resources spent on expanding it are wasted.*

4.3 Property Constraints: Early Filtering

The default construction, like the query plans used by SOA graph DBMS, applies a "difficult" property constraint only after fully materializing candidate paths. To eliminate doomed intermediate paths early, we must identify all negatively stable constraints and maintain path state that enables early detection when any of them becomes `false`. To this end we propose *selective aggregates*, which accumulate state along a path in a dictionary D of key-value pairs. They also expose a Boolean function that identifies when a negatively stable constraint turns to `false`.

Since all constraints in Q_A are negatively stable (Example 4), we can use them all for early filtering. Consider the dictionary instance of intermediate path x. For *timestamp monotonicity*, D contains key `last_time` whose value is the timestamp of the last edge (transaction) in x. When considering expanding x with an edge e, the resulting path $x \circ e$ is doomed if $e.\texttt{time} \leq x.D[\texttt{last_time}]$. For the *max-min amount difference*, D maintains the greatest and smallest transaction amounts over all of x's edges in $D[\texttt{max_amount}]$ and $D[\texttt{min_amount}]$, respectively. Then $x \circ e$ is doomed if $\max(e.\texttt{amount}, x.D[\texttt{max_amount}]) - \min(e.\texttt{amount}, x.D[\texttt{min_amount}]) > U$. Note that if the condition was a lower bound on the max-min difference, then early filtering would not be possible: Even if the gap is below the threshold for x, adding more edges could widen it. Lastly, early filtering on *trail semantics* can be enabled by maintaining the set of edge ids in $D[\texttt{edge_ids}]$: $x \circ e$ is doomed if $e.\texttt{id} \in x.D[\texttt{edge_ids}]$. This is an example where a dictionary value is not a scalar.

For Q_B, $D[\texttt{last_time}]$ can analogously be used to check if $e.\texttt{time} - x.D[\texttt{last_time}]$ falls into the appropriate range. But now the specific range depends on the transition in the label pattern (domestic to domestic vs domestic to foreign transaction). Hence in addition to intermediate path x and candidate edge e, we must also take into account the NFA transition. This is easy to specify in RECAP as the left part of Figure 5 illustrates. Each NFA transition is annotated with 2 functions: `update_d` and `is_viable_d`. The former updates the dictionary state, while the latter checks viability. The complete functions are obtained in a purely syntactic manner by copying the individual code blocks into the corresponding branch of a CASE statement as shown on the right in Figure 5. Note that in contrast to `is_viable_d`, the behavior of `update_d` here does not depend on the NFA transition, i.e., the CASE statement in the complete function is unnecessary. Like for a standard aggregate, dictionary state needs to be initialized and a final output may need to be extracted in the end:

[3] We only discuss constraints on edge properties; vertex properties are analogous.

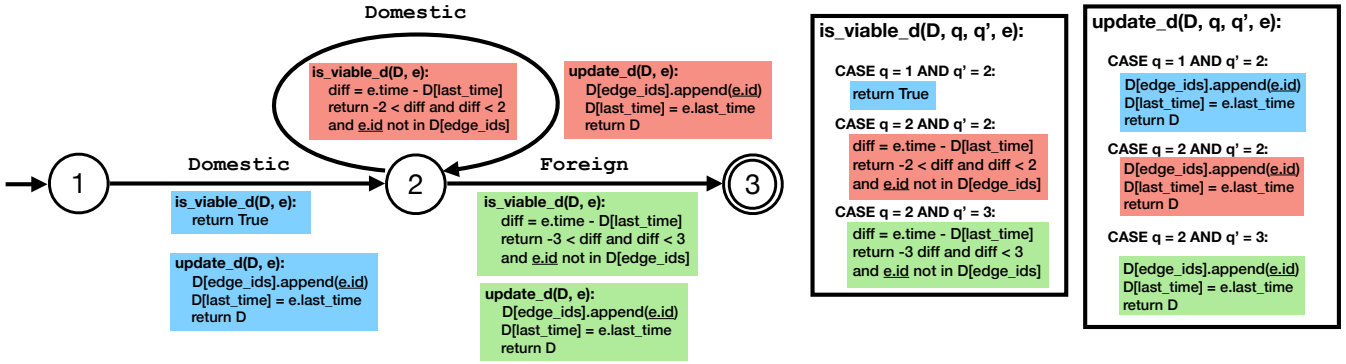


Figure 5: NFA with `is_viable_d` and `update_d` calls for Q_B.

Definition 8 (Selective Aggregate). *For an edge type E and an NFA with state set $\mathfrak{Q}$, a* selective aggregate *is a tuple consisting of*

- *Dictionary D of key-value pairs. A value is a scalar or an object, e.g., a list.*
- *Function* `init_d` $: \emptyset \to D$ *to initialize the dictionary.*
- *Function* `update_d` $: D \times \mathfrak{Q} \times \mathfrak{Q} \times E \to D$ *to update the dictionary.*
- *Function* `finalize_d` $: D \to D$ *to convert the dictionary content to a new dictionary for returning it as the final result.*
- *Function* `is_viable_d` $: D \times \mathfrak{Q} \times \mathfrak{Q} \times E \to \{\texttt{true}, \texttt{false}\}$ *to enforce dictionary-content-dependent constraints on state transitions.*
- *Function* `is_viable_d_final` $: D \to \{\texttt{true}, \texttt{false}\}$ *to enforce the final dictionary content satisfies the constraints.*

The default construction (Section 4.2) is a special case of a selective aggregate where `init_d` returns an empty list (implemented by a dictionary with a single key whose value is the list), `update_d` appends the edge tuple to the list, `is_viable_d` returns `true`, and `is_viable_d_final` evaluates φ on the list.

4.4 ReCAP Definition and Semantics

Definition 9 (ReCAP). *The ReCAP for a path query $Q = (S, R, \varphi)$ over a property graph G is a pair (N, A), consisting of the NFA N for R and a selective aggregate A for φ and N.*

We provide a constructive definition of ReCAP's semantics via the SQL query shown in Listing 3. It is a simple recursive join query that calls the functions of the selective aggregate. Thus, in addition to providing a clean abstraction for specifying early filtering opportunities via an NFA and a selective aggregate, a major benefit of ReCAP is that it does not require any database engine modifications or SQL language extensions. All customization for a given query is encapsulated in the transitions table T (for the regex R) and the functions of the selective aggregate (for property constraint φ; underlined in the listing). Note how the WHERE clause of the recursive member enforces not only the label pattern, but also enables early filtering via the `is_viable_d` call. The values of s, q_0, and $\mathfrak{Q}_F$ are passed as parameters from the ReCAP definition.

How do we implement dictionary D? Some queries may need multiple key-value pairs or complex value types, like the list of edge IDs for trail semantics. Since modern databases support JSON columns, by default we add D as a JSON column to Paths.

Listing 3: SQL code implementing any ReCAP as a simple recursive join query that calls the 5 user-defined functions of its selective aggregate (underlined).

```
WITH RECURSIVE Paths AS (
    SELECT s AS v,
           q0 AS q,
           init_d() AS D
    UNION ALL
    SELECT E.dst AS v,
           T.to_state AS q,
           update_d(P.D, P.q, T.to_state, E.*) AS D
    FROM Paths P
    JOIN Edges E ON P.v = E.src
    JOIN Transitions T ON P.q = T.from_state
    WHERE T.label = E.label AND
        is_viable_d(P.D, P.q, T.to_state, E.*)
)
SELECT v, finalize_d(D)
FROM Paths
WHERE q IN 𝔔F AND is_viable_d_final(D)
```

5 ReCAP Creation and Properties

We show how to construct selective aggregates for ReCAP, then study ReCAP's expressiveness and establish complexity bounds for query evaluation.

5.1 Creating Selective Aggregates

Given a path query $Q = (S, R, \varphi)$, it is the user's responsibility to identify the appropriate ReCAP. Since the NFA is generated automatically from R, this boils down to choosing or creating the selective aggregate. The process is similar to standard aggregates, e.g., like recognizing that AVG is algebraic [22] and can be incrementally maintained as a pair (SUM, COUNT). **In the absence of such insights, the user falls back to the default construction (Section 4.2), which is no worse than what SOA systems offer.**

Fortunately, as Q_A and Q_B illustrate, many practical opportunities for early filtering are easy to identify. Then, like for standard aggregates, one defines the relevant state and the functions to initialize (`init_d`), accumulate (`update_d`), and finalize (`finalize_d`)

it. For early filtering, the user must identify the relevant negatively stable constraints, define them in a general way so that they apply to paths of any length, and then check them in `is_viable_d` and `is_viable_d_final`. Here ReCAP's automaton-based formalism provides an additional route for proving correctness based on the insight that a path is an answer iff it reaches an accepting state in the NFA and all `is_viable_d` calls along the way return `true`; as does `is_viable_d_final`.

When path queries can be supported by a **factorized** ReCAP, i.e. the selective aggregate is independent of the NFA state transitions, then a library of common cases can be created similar to user-defined functions (UDFs) in SQL. For example, SUM is not specific to a given table or query, but a general function applicable to any list of numbers. We now describe such constructions for common path constraint types. Let $p' = v_0 e_1 v_1 \cdots e_j v_j$ be a path, $p'.D$ its dictionary, and e an edge considered for appending to p'.

Negatively stable constraint on individual edges. A common type of constraint C involves adjacent path edges; formally $\forall i < j : C(e_i, e_{i+1})$. Here the selective aggregate stores the property value of the last edge e_j of p' in D.`last`, with `is_viable_d` returning $C(D.\texttt{last}, e)$. This generalizes easily to any constraint over any subset of path edges.

Example 7 (Time Monotonicity). *With* $C(e_i, e_{i+1}) \equiv \forall i < j : e_i.\texttt{time} < e_{i+1}.\texttt{time}$, *set* $D[\texttt{last_time}] = e_j.\texttt{time}$. *For candidate edge* e, `is_viable_d` *checks* $D[\texttt{last_time}] < e.\texttt{time}$.

Example 8 (Trail Semantics). *With* $C(e_1, \ldots, e_j) \equiv \forall i < j : e_{i+1}.\texttt{id} \notin \{e_1.\texttt{id}, \ldots, e_i.\texttt{id}\}$ *set* $D[\texttt{edge_ids}] = \{e_1.\texttt{id}, \ldots, e_j.\texttt{id}\}$. *For candidate edge* e, `is_viable_d` *checks* $e.\texttt{id} \notin D[\texttt{edge_ids}]$.

Negatively stable constraint on path aggregates. Another common type of constraint involves a distributive aggregate [22] of certain property values along the path; formally $\forall i \le j : C(\text{agg}(e_1, \ldots, e_j))$, where agg is an aggregate function. Here the selective aggregate stores $\text{agg}(e_1, \ldots, e_j)$ in $D.\text{agg}$,[4] with `is_viable_d` returning $C(\text{agg}(D.\text{agg}, e))$. It is straightforward to generalize this to constraints involving multiple aggregates and to algebraic aggregates [22]. This corresponds to the fold-style aggregates supported by Cypher's list processing.

Example 9 (Upper Bound on Max-Min Amount Difference). *With* $C(\max(e_1, \ldots, e_i), \min(e_1, \ldots, e_i))$ *defined as*

$$\forall i \le j : \max(e_1.\texttt{amount}, \ldots, e_i.\texttt{amount}) - \min(e_1.\texttt{amount}, \ldots, e_i.\texttt{amount}) < U$$

set

$$D.\texttt{max_amount} = \max(e_1.\texttt{amount}, \ldots, e_j.\texttt{amount})$$
$$D.\texttt{min_amount} = \min(e_1.\texttt{amount}, \ldots, e_j.\texttt{amount}).$$

For candidate edge e, `is_viable_d` *checks*

$$\max(D.\texttt{max_amount}, e.\texttt{amount}) - \min(D.\texttt{min_amount}, e.\texttt{amount}) < U.$$

Clearly, not all constraints on incrementally maintainable path aggregates are negatively stable. Consider a *lower* bound L on the max-min difference of transaction amount along a path. While max and min can be maintained as before, their difference can increase as more edges are added. Hence even if a path prefix violates the lower bound, some of its extensions could satisfy it. For such constraints, early filtering is not possible.

[4] We slightly overload notation, using agg to refer to the function and its name.

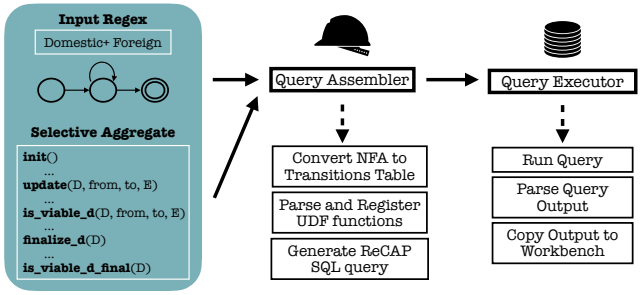


Figure 6: ReCAP workbench pipeline. The user specifies label regex and selective aggregate. The system generates the NFA, assembles the ReCAP SQL query with the transitions table and UDFs, and executes the query.

What if the ReCAP is not factorized, such as the timestamp difference in Q_B? Then the selective aggregate must take into account the NFA's state transition via a CASE statement as shown for the `is_viable_d` and `update_d` functions in Figure 5. (The latter actually does not need the CASE statement, because the update behavior is independent of the state transition.) To this end, we simply add from and to state of the NFA to the input of `is_viable_d` and `update_d`. A factorized ReCAP ignores these states.

By using the same signature for all selective aggregates, no matter if the ReCAP is factorized or not, selective aggregates *compose naturally*. When using multiple property constraints, their dictionaries are merged (possibly requiring renaming of keys to ensure uniqueness) and their functions are conjoined. The result is itself a selective aggregate, with no change to the query structure.

5.2 Implementing and Executing a ReCAP

To author a ReCAP query, the user specifies a regular expression over the edge-label alphabet and writes the code (in Python) for the 5 functions that define the selective aggregate. The system converts the regex to an NFA via Thompson's construction, materializes it as a Transitions relation, injects the Python functions as relational DBMS UDFs (we use DuckDB in our case), and executes the assembled query. We developed an interactive browser-based workbench whose architecture is shown in Figure 6. It supports two modes: a *default* mode where the user provides only defines `is_viable_d_final` and `finalize_d`, and an *early-filtering* mode exposing all five functions (`init_d`, `update_d`, `is_viable_d`, `finalize_d`, `is_viable_d_final`). In early-filtering mode, the workbench can generate `CASE`-statement stubs from the current NFA transitions, so the user only needs to fill in the transition-specific logic. The compilation to standard ReCAP SQL (Listing 3) is fully implemented; automating the optimization layer of Section 6 (dictionary flattening and function inlining) is left for future work.

5.3 Expressiveness

Previous work on generalizing path queries beyond regex on labels (and their matching automata models with memory) focused on

carefully navigating the tradeoff between expressiveness and time complexity (Section 8), showing that even small extensions can quickly lead to NP hardness [21]. Our design philosophy differs in that the goal of ReCAP is to provide a mechanism for early filtering on property constraints for *all path queries, even hard ones*. Stated differently, the main design goal of ReCAP is not to limit expressiveness (and thus complexity), but to **provide a general mechanism for exposing structure that enables early filtering on property constraints**. This is in line with SQL's evolution toward Turing-completeness: if the user wants to write a hard query, then, instead of forcing them to look elsewhere, offer the most efficient implementation.

Theorem 5.1 (Expressiveness). *ReCAP supports any regex R over vertex and edge labels and any constraint over both properties and labels of vertices and edges.*

This follows directly from the default construction (Section 4.2). In fact, even the label regex R could be enforced by φ on the complete path candidate. Clearly, for early filtering it is beneficial to expose structure via the NFA states and the label checks and `is_viable_d` calls along transitions.

Observation 1 (Regret-Free Optimization). *When φ is not negatively stable, then the default construction (Section 4.2) produces a query plan that reduces to the execution strategy of current SOA systems: apply early filtering using the label regex R, then check property constraints on the complete path candidates.*

5.4 Complexity Analysis

Since "return all paths in G" is a valid path query, worst-case time complexity cannot be lower than the number of possible paths $|\Pi(G)|$. The same applies to the SOA graph and relational DBMS. For more fine-grained results, we therefore analyze complexity *parameterized* on relevant factors: Let t_v denote the maximum time for a `is_viable_d` or `is_viable_d_final` call, t_u the maximum time for a `update_d` or `finalize_d` call, ℓ the maximum path length[5], P_ℓ^{late} the number of paths of length up to ℓ explored by the default construction, and P_ℓ^{early} the number explored under early filtering. Since early filtering removes doomed intermediate paths, $P_\ell^{\text{early}} \leq P_\ell^{\text{late}}$, potentially with a huge gap.

Proposition 5.2 (Time Complexity). *The time complexity of Listing 3 is $O\left(P_\ell^{\text{late}} \cdot (\ell + t_v + t_u)\right)$ for the default construction and $O\left(P_\ell^{\text{early}} \cdot \ell \cdot (t_v + t_u)\right)$ for early filtering.*

Proof. Consider a path p of length up to ℓ explored by Listing 3. With the default construction, for each of p's edges, this edge is appended to $p.D$, taking total time $O(\ell)$. In the end, `is_viable_d_final` is called and then `finalize_d` to generate the output tuple. With early filtering, each time an edge is added to the path, `is_viable_d` and `update_d` are called. In the end, `is_viable_d_final` and `finalize_d` are called.

In addition, we need to account for the join cost to assemble p. We follow common practice and assume that with hash indices on Edges (with `src` vertex as the key) and Transitions (with combined key (`from_state`, `label`)), the total join time is linear in the number of paths explored. The corresponding query plan iterates through the Paths relation: for each intermediate path, it looks up edges adjacent to its end vertex in the Edges index and candidate transitions in the Transitions index, producing the corresponding longer intermediate paths. □

[5]It could be ∞ when edges and vertices are allowed to repeat and the graph has cycles.

ℓ	Runtime (s)	UDF (s)	Non-UDF (s)	UDF %
2	0.02	<0.01	0.02	≈0%
3	0.30	0.19	0.11	63%
4	10.13	7.85	2.28	77%
5	385.09	375.20	9.89	97%
6	13 526.79	13 315.16	211.63	98%

Table 1: Execution time breakdown for the Python UDF baseline across increasing path lengths. UDF time aggregates the 5 ReCAP functions as reported by `EXPLAIN ANALYZE`. Non-UDF time covers index scan, hash joins, and CTE management.

As illustrated by Q_A and Q_B, the time complexity of the functions of the selective aggregate is often proportional to dictionary size (or lower, e.g., constant time for appending a new edge ID to `edge_ids`):

Corollary 5.3. *If $t_v, t_u \in O(\ell)$, then the time complexity of the default construction is $O\left(P_\ell^{\text{late}} \cdot \ell\right)$. If $t_v, t_u \in O(|D|)$ and dictionary size $|D|$ of the selective aggregate is upper-bounded by a constant (i.e., independent of path length), then the time complexity for early filtering is $O\left(P_\ell^{\text{early}} \cdot \ell\right)$.*

6 ReCAP Performance Optimization

The ReCAP SQL template (Listing 3) provides an elegant way to enable early filtering for any negatively stable constraint, achieving several orders of magnitude speedup over SOA graph and relational databases (see Section 7.3). We now propose more low-level optimizations that squeeze out another 1 or 2 orders of magnitude speedup, especially for challenging queries.

Relational DBMS are the most efficient approach for Select-Project-Join (SPJ) queries over flat tabular data, but the selective aggregate introduces additional overhead for calling UDFs and for manipulating a JSON-encoded dictionary. Our measurements on various path queries and input graphs indicate that this overhead can dominate running time, especially for longer paths. Table 1 shows representative results, here for query Q_3, limited to paths of at most ℓ edges, for different ℓ, over a graph with 84.3 million edges generated by LDBC's Datagen7.6 (see Section 7 for details). We implemented the selective aggregate's UDFs in Python and enforced single-threaded execution to best isolate UDF overhead from other costs. Already for $\ell = 3$, UDF overhead accounts for 63% of the time, rapidly approaching 98% for $\ell = 6$. This cost is attributable to two compounding factors: JSON manipulations when accessing and updating dictionary D (80–92% of UDF time), and the per-call overhead of context switching between SQL and Python.

To reduce this overhead, we propose integrating the operations of the selective aggregate natively into the query engine via two

```
WITH RECURSIVE Paths AS (
  SELECT
    s as v,
    q0 as q,
    NULL::TIME as last_time,                          <-- init_d
    [ ] as edge_ids,
    0 as path_length
  UNION ALL
  SELECT
    E.dst as v,
    T.to_state as q,
    E.time as last_time,                              <-- update_d
    list_append(P.edge_ids, E.edge_id) as edge_ids
    p.path_length + 1 as path_length
  FROM
    Paths P, Edges E, Transitions T
  WHERE P.v = E.src AND T.label = E.label AND P.q = T.from_state
  AND CASE                                            <-- is_viable_d
     WHEN P.q = 1 AND T.to_state = 2 THEN TRUE
     WHEN P.q = 2 AND T.to_state = 2 THEN
        -2 <= (E.time - P.last_time) AND
        (E.time - P.last_time) <= 2 AND
        NOT list_contains(P.edge_ids, E.id)
     WHEN P.q = 2 AND T.to_state = 3 THEN
        -3 <= (E.time - P.last_time) AND
        (E.time - P.last_time) <= 3 AND
        NOT list_contains(P.edge_ids, E.id)
     ELSE FALSE
   END)
SELECT v, edge_ids                                    <-- finalize_d
FROM Paths
WHERE q IN accepting_states AND True                  <-- is_viable_d_final
```

Figure 7: Fully optimized code for the Example 5 RECAP.

complementary transformations: dictionary flattening and function inlining. They convert Listing 3 into SQL code free of JSON operations and UDF calls.

6.1 Dictionary Flattening

Our first optimization applies *dictionary flattening* to reduce the cost of traversing and manipulating nested data—and ideally eliminate the need for JSON encoding altogether. This operation removes 1 level of nesting from the dictionary D of the selective aggregate. For each key-value pair (K, V) in D, we add a column K to the Paths relation to store V there. When a value is a scalar expressible with a built-in SQL type, JSON operations are eliminated entirely for that key. For list-style values we use SQL array types where supported (e.g., DUCKDB); otherwise the value remains in JSON but with one less level of nesting.

For Q_B, `last_time` can be encoded with the appropriate numerical or date type. For the list `edge_ids`, we use an array type in DuckDB, or represent it in JSON in a DBMS with no array type.

6.2 Function Inlining

Function inlining consists of 2 steps. First, we rewrite the UDF in SQL. Then we replace each UDF call in Listing 3 with its function body. This complements dictionary flattening: when dictionary keys turn into flat columns, the inlined SQL accesses them directly as scalar values.

Figure 7 shows the fully optimized SQL for Q_B after applying both transformations. In the anchor, `init_d` becomes direct column initialization (`NULL` for `last_time`, empty list for `edge_ids`). In the recursive member, `update_d` becomes scalar assignments and list appends, while `is_viable_d` becomes a `CASE` statement branching on the NFA states q and q'. In the outer query, `finalize_d` reduces to column selection and `is_viable_d_final` to a constant `TRUE`. Since all cases in `update_d` execute the same code for Q_B, the `CASE` is dropped there. For `is_viable_d` the `CASE` must be preserved because the timestamp range depends on the NFA transition (2 days for domestic-to-domestic, 3 days for domestic-to-foreign). For factorized RECAPs where property constraints do not depend on NFA state, all `CASE` expressions collapse to unconditional expressions.

After flattening and inlining, the query contains no UDF calls or JSON operations. The database engine now deals with a standard recursive SPJ query and can apply its full repertoire of optimizations.

Graph	$\|V\|$	$\|E\|$
BITCOIN	5k	35k
Metaverse	1.3k	78k
Reddit	35k	286k
LDBC100 (P-K-P)	448k	19.9M
Datagen-7.6	754k	84.3M
Datagen-7.7	13.1M	53.7M

Table 2: Datasets used in the experiments.

7 Experiments

We study RECAP's effectiveness for early filtering on property constraints, comparing it to SOA graph and relational DBMS.

7.1 Setup and Methodology

Data and Queries. We work with real and synthetic data commonly used for path queries (Table 2), with real data from SNAP [29–32], Kaggle [12], and LDBC [25]. Due to a lack of established benchmarks for path queries with property constraints, we create a mix of queries with varying opportunities for early filtering as summarized in Table 3.

Q_1 analyzes financial data where directed edges represent transactions with labels (`transfer`, `purchase`, `sale`, `phishing`, `scam`) and properties such as amount, risk score, timestamp, and geographic region. The query contains all types of constraints: label regex, negatively stable constraints on individual edges and path aggregates, and property constraints that are not negatively stable. The regex specifies a transaction chain that looks normal at first, then turns fraudulent. The property constraints ensure (i) increasing timestamps for chronological event sequence [61], (ii) a single jurisdiction (region), (iii) a limited risk-score range for the "normal" initial transactions to capture a consistently-profiled chain [44], (iv) a high risk for the last "normal" transaction, reflecting its role as the gateway to fraudulent activity [9], and (v) a high total amount across the path to filter out financially insignificant chains.

Q_2 has a property constraint that is not negatively stable: the repeating color may appear on the very last edge of the path. Since the given graphs do not have edge colors, we control query selectivity through the number of distinct color values. In the presented experiments, we add a color that is selected uniformly at random

Query	Description
Q_1	Label regex: (`transfer` | `purchase` | `sale`)$^+$ · (`phishing` | `scam`)$^+$. Property constraints: chronological timestamps, same region, risk score range ≤ 20 (in normal prefix), last normal edge risk ≥ 40, total amount ≥ 1000.
Q_2	Property constraint: there exist two adjacent edges that have the same color.
Q_3	Property constraint: strictly increasing numeric edge property (e.g., weight, timestamp).
Q_4	Property constraint: earliest and latest edge timestamp along the path does not exceed two weeks.

Table 3: Queries used in the experiments. All of them use trail semantics, i.e., no edge can be traversed more than once.

from 5 possible values. Q_3 is similar to Q_1, but only uses a monotonicity constraint (which is negatively stable) along the path to enforce strict temporal causality of events. Q_4 enforces temporal cohesion (which is negatively stable) across the path when connecting interactions in a social network.

For each query, we randomly selected start vertices from each inter-quartile range of the out-degree distribution (low: < 25%, medium: 25–75%, high: > 75%) to include everything from "poorly" to "well" connected vertices. No matter the start vertices, we observed the same trend. Hence we show representative results with the following settings: high-degree start vertices for Q_1, medium-degree for Q_2, Q_4, and low-degree for Q_3.

Output and Success Measures. To compare query time independent of output-transfer strategy, all queries *count* the number of result paths. We verified that all query plans enumerate paths before computing the count, with no counting-induced shortcuts. We report two measures: running time from query submission until the entire output is returned and *total* number of candidate paths explored. For systems without early filtering, the latter equals the number of paths matching regex R with φ = `true`. For each query-graph pair, we search for paths of length up to ℓ for $\ell \in \{2, \ldots, 10\}$, running each configuration 5 times after one warm-up run. Results across runs were within 5% of each other and consistent for all systems, therefore we simply report the median. A timeout of 2 hours was enforced per run.

Systems and Environment. We compare ReCAP implemented in DuckDB (version 1.4.1) against DuckDB without ReCAP, and System X, a SOA commercial relational DBMS known for its enterprise-level performance and reliability. In addition, we compare against the leading graph DBMS Neo4j [46] (version 5.24.1), Memgraph [42] (version 3.6.1), and Kùzu [15] (version 0.11.2). For the relational DBMS, we create a standard adjacency index on the Edges relation in advance, which is a hash index on the `src` column. The graph DBMS maintain an analogous index as well. All experiments were executed on a server with a dual Intel Xeon Gold 5218 CPU (2.30 GHz, 32 physical cores) and 768 GB RAM.

7.2 Performance Impact of Optimizations

We implemented ReCAP in SQL in DuckDB as discussed in Section 6, comparing the standard version (Listing 3) against the fully optimized one that applies dictionary flattening and function inlining where possible (Figure 7). We refer to them as ReCAP-Standard and ReCAP-Optimized, respectively. Both rely on a standard SPJ query with recursion to expand all paths to the desired length ℓ. The ReCAP functions needed by ReCAP-Standard are implemented as UDFs in Python. Both versions were executed via the DuckDB-Python API.

Figure 8 shows representative results, demonstrating that ReCAP-Optimized is the clear winner, regardless of query, graph, or path length. The speedup of up to 346× stems from the elimination of ReCAP-Standard's huge number of function calls that require loading and parsing the JSON representing the dictionary.

For Q_1 applied to the Metaverse transaction graph, the speedup is 152× for $\ell = 10$. For Q_2 the speedup is 91× for $\ell = 4$. For greater values of ℓ ReCAP-Standard timed out. This is not surprising, because despite the small input size, there is a combinatorial number of intermediate results. In contrast, Q_3 and Q_4 can apply early filtering and thus scale to larger graphs and longer paths. For Q_3, at $\ell = 4$ we observe 156× speedup. For Q_4 applied to Person-Knows-Person table (P-K-P) edges of the LDBC Social Network benchmark (scale factor 100), the speedup reaches 8× at $\ell = 4$ and widens to 346× at $\ell = 10$. The results confirm the effectiveness of our optimizations, therefore we only include the optimized version of ReCAP in the experiments below.

7.3 Running-Time Comparison

Summary. ReCAP implemented in standard SQL in DuckDB outperforms DuckDB without ReCAP, System X, and the specialized SOA graph DBMS by orders of magnitude. The gap widens as path length increases. As we show in Section 7.4, this advantage stems from pushing edge property constraints deep into path exploration, reducing intermediate paths generated and enabling ReCAP to scale to paths roughly twice as long as the competition.

Evaluating Q_1 in Neo4j, Memgraph, and Kùzu exposes two inefficiencies. First, limited regex support forces decomposition into two segments joined at an intermediate vertex, inducing a plan where matches for each segment are found separately and then joined. Second, all property constraints are applied at the end, yielding huge intermediate results: for $\ell = 4$, the normal prefix alone produces over $25M$ candidate paths, of which only $5k$ are answers. In contrast, ReCAP maintains path state (timestamp, region, risk range, last risk, cumulative amount), filtering doomed paths early. As Figure 9 shows, this yields for $\ell = 4$ an 82940× speedup over the graph DBMS and 1265× over DuckDB without ReCAP. The advantage of DuckDB without ReCAP over the graph DBMS comes from *our implementation of the regex* as a join with the transitions table. Nonetheless, all competitors share the same late-filtering approach for property constraints. System X, despite also using a

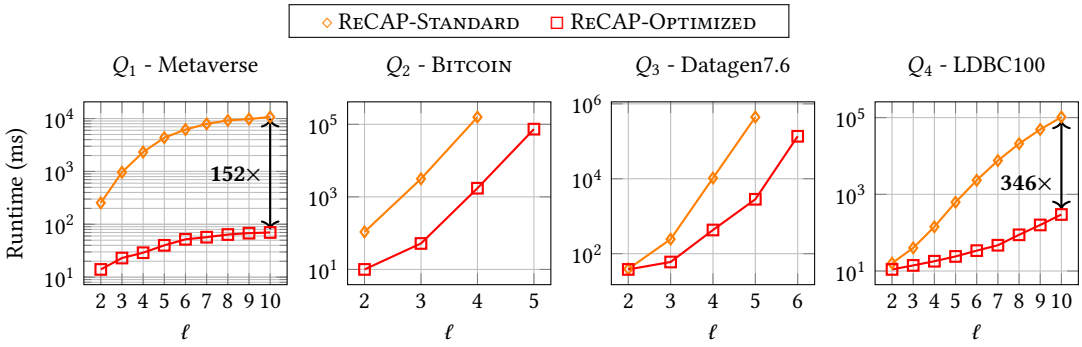


Figure 8: Running-time comparison between ReCAP-Standard vs ReCAP-Optimized

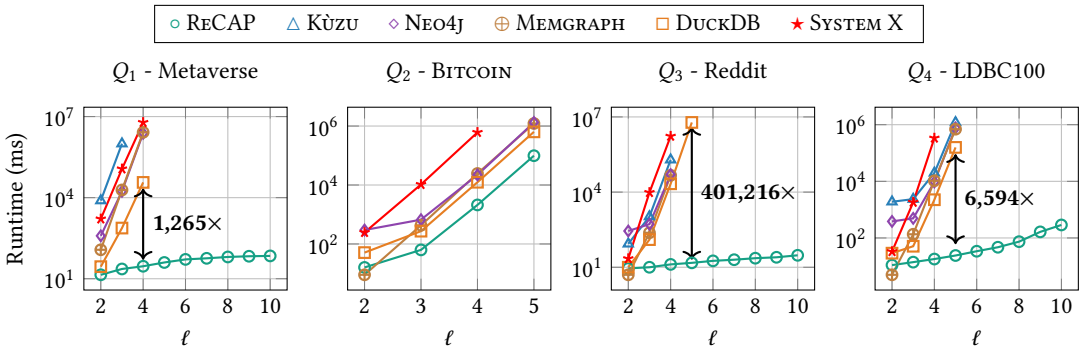


Figure 9: Running-time comparison across different queries and graphs. The competitors behave very similar to each other and scale poorly to larger ℓ due to the huge number of intermediate results. ReCAP's ability to push property constraints deep into path exploration lets it scale to more than double the path length. The exception is Q_2, which does not admit early filtering.

join for the regex, is further penalized by its lack of native array types, requiring management of JSON-encoded path state.

Since Q_2 does not admit early filtering, all systems share the same query plan. Yet ReCAP still achieves a speedup for larger outputs by piggybacking color checks on path exploration rather than doing them in a separate post-processing pass. Specifically, ReCAP tracks `last_color` and a Boolean `completed`, checked in `is_viable_d_final`. For $\ell = 5$, this yields a 12× speedup over the graph DBMS and 6× over DuckDB without ReCAP, illustrating that incremental constraint evaluation reduces overhead even when early filtering is not possible.

Q_3 and Q_4, like Q_1, admit early filtering. Figure 9 shows competitors timing out at or before $\ell = 6$; at $\ell = 5$, ReCAP is 401216× and 6594× faster than the best competitor DuckDB.

7.4 Intermediate Result Size Comparison

To understand the root cause of ReCAP's running-time improvement, we report intermediate path counts in Figure 10. It is clear that intermediate result cardinality closely mirrors the running-time trend. Since the difference between ReCAP and plain DuckDB is early filtering, intermediate cardinality directly explains the running-time gap between them. By inspecting query plans, we confirmed that all competitors first generate candidate paths based solely on the label regex (if specified) and then post-process them with the property constraints. ReCAP changes this behavior for DuckDB by pushing property constraints deep into path exploration. The graph DBMS currently offer no comparable mechanism: In their `MATCH` clause, Cypher, GQL, and SQL/PGQ cannot express property constraints like those in our queries [20, 21], forcing evaluation over complete paths via "list processing."

While Neo4j and Memgraph enforce trail semantics (no repeated edge) during the regex matching phase, Kùzu, System X and DuckDB enforce it afterward, applying walk semantics initially (allows repeat vertices and edges). As Figure 10 shows, the cardinality difference between early and delayed enforcement of trail semantics is negligible here. The reasons are (1) sparseness of the graph, (2) small path length (since the systems time out for longer paths), and, for Q_1, (3) the selectivity of the regex that makes loops almost impossible for short paths.

For Q_1, at $\ell = 4$, the competitors generate over *3.5 million* intermediate paths, whereas ReCAP produces only 324, which is remarkably close to the final output of 264. Since Q_2 admits no early pruning, all systems produce the same intermediate paths. The trail vs. walk distinction is again negligible: at $\ell = 3$, trail-enforcing systems produce 163,710 paths vs. 163,756 for the others; at $\ell = 5$, it is 372,223,533 vs. 372,750,847. For Q_3 and Q_4, the pattern mirrors Q_1. At $\ell = 4$, ReCAP generates 307 intermediate paths for Q_3, versus 20,873,911 for Neo4j and Memgraph and 20,876,598 for Kùzu, System X and DuckDB; at $\ell = 5$, the competitors produce over a million times as many! For Q_4, the competitors generate 3,378,749 and 203,556,023 intermediate paths at $\ell = 4$ and $\ell = 5$, respectively, compared to 1,160 and 4,792 for ReCAP.

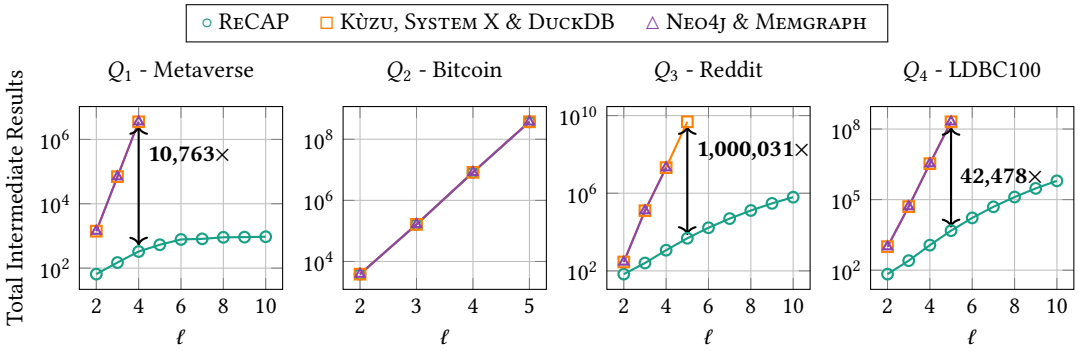


Figure 10: Total number of intermediate paths generated across different queries and datasets. Neo4j and Memgraph follow the same query plan, producing the same number of intermediate paths based on trail semantics. Similarly, Kùzu, System X and DuckDB produce the same number of intermediate paths, but based on walk semantics, enforcing trail semantics afterward.

8 Related Work

Path queries over graphs have been extensively studied, with Regular Path Queries (RPQs), which match edge-label sequences against regular expressions [5, 11], serving as the foundational path pattern. The dominant evaluation paradigm for RPQs is the product-graph construction—proposed by [43]—where the input graph is traversed jointly with an automaton representing the query. This approach underlies a wide range of systems and techniques [3, 4, 6, 8, 28, 34, 49, 53, 55, 56]. Some work extended RPQs to incorporate data (property) constraints, such as comparisons and aggregates [16, 36, 37]. Said work has primarily focused on richer automata models such as register automata [27], parametric automata [33], variable automata [54], and Parikh automata [17], focusing on the tradeoff between expressiveness and complexity. In contrast to previous work, the goal of ReCAP is not to hit a certain expressiveness-complexity sweetspot. (It can support any constraint over properties!) Instead, ReCAP provides a mechanism that supports exposing relevant structural properties for *any path query*, including those with both label and property constraints, so that a relational DBMS can efficiently execute these queries.

Recent work on graph query languages and standards has highlighted the growing importance of combining label patterns with property constraints [35]. However, due to limitations in expressiveness of pattern-matching clauses [20], these systems evaluate "difficult" property constraints in the end over materialized paths, often using list-based constructs [21]. This leads to poor scalability, as SOA graph and relational DBMS are not able to identify opportunities for early filtering and thus produce large intermediate results. Our work directly addresses this gap by exposing negatively stable path constraints, thus enabling early filtering during traversal rather than after full path materialization.

A complementary line of work studies efficient path enumeration and evaluation strategies for paths, including optimizations for path modes such as shortest paths, simple paths, or trails [13, 40]. These approaches focus primarily on topological constraints and enumeration efficiency, but do not address the general problem of pushing property constraints deep into path exploration. Other techniques study query decomposition and selective exploration, such as splitting RPQs based on rare edge labels [28, 34, 47, 48] or using automata-based index structures [59, 60]. While these methods reduce the search space under specific conditions, they do not provide a general abstraction for expressing and exploiting early filtering across heterogeneous constraints, especially those involving aggregates or cross-edge dependencies.

Finally, a line of work explores relational approaches to graph querying, leveraging recursive queries and join processing in relational DBMS. Early efforts formalized RPQs using relational algebra by adding transitive closure [39] or studying graph-query languages and mapping them to SQL [41, 58]. Recent research showed that relational engines can outperform specialized graph DBMS on RPQs and their extensions (conjunctions and unions of RPQs) [10, 14, 57]. Systems such as GRFusion [24], GQ-Fast [38], GraphflowDB [26], and DuckPGQ incorporate graph abstractions into relational engines. Some support limited forms of predicate pushdown, but would fail to do so for more complex conditions like in Examples 1 and 5 (or the queries in Table 3). Those approaches are restricted to simple predicates or specific query classes and do not address the general challenge of pushing complex, stateful path constraints into recursive evaluation. ReCAP provides a unifying abstraction that captures a broad class of constraints amenable to incremental evaluation and compiles them into relational execution plans with effective early filtering.

9 Conclusions and Future Work

We demonstrated that SOA graph and relational DBMS struggle to push property constraints deep into path-query processing. To address this issue, we introduced the notion of negatively stable constraints and proposed ReCAP as a clean abstraction for specifying the corresponding early filtering opportunities. This reduced intermediate result cardinality by a factor of up to 1 million and running time by a factor of up to 400,000, even for simple queries and small graphs. When a query does not admit early filtering, or when one cannot identify negatively stable constraints, ReCAP behaves like the SOA competitors, thus providing "regret-free" optimization. Exciting future directions include more advanced optimizations, e.g., based on the most selective fragments of a path query, and an extension to more general path queries, e.g., those that rank answers. We are also interested in designing an automated method for generating the optimized SQL code from a ReCAP instance specified in a high-level language.

References

[1] [n. d.]. SQL/PGQ. https://duckpgq.org/documentation/sql_pgq/. [Accessed 1-Nov-2025].

[2] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. 2017. Foundations of modern query languages for graph databases. *ACM Computing Surveys (CSUR)* 50, 5 (2017), 1–40.

[3] Diego Arroyuelo, Aidan Hogan, Gonzalo Navarro, and Javiel Rojas-Ledesma. 2022. Time-and space-efficient regular path queries. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 3091–3105.

[4] Jorge Baier, Dietrich Daroch, Juan L. Reutter, and Domagoj Vrgoč. 2017. Evaluating Navigational RDF Queries over the Web. In *Proceedings of the 28th ACM Conference on Hypertext and Social Media* (Prague, Czech Republic). Association for Computing Machinery, New York, NY, USA, 165–174. doi:10.1145/3078714.3078731

[5] Pablo Barceló Baeza. 2013. Querying graph databases. In *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGAI symposium on Principles of database systems*. 175–188.

[6] Georgiy Belyanin and Semyon Grigoriev. 2024. Single-Source Regular Path Querying in Terms of Linear Algebra. *arXiv preprint arXiv:2412.10287* (2024).

[7] Lars Brenna, Alan Demers, Johannes Gehrke, Mingsheng Hong, Joel Ossher, Biswanath Panda, Mirek Riedewald, Mohit Thatte, and Walker White. 2007. Cayuga: a high-performance event processing engine. In *Proceedings of the 2007 ACM SIGMOD international conference on Management of data (SIGMOD '07)*. Association for Computing Machinery, New York, NY, USA, 1100–1102. doi:10.1145/1247480.1247620

[8] Marco Bucchi, Alejandro Grez, Andrés Quintana, Cristian Riveros, and Stijn Vansummeren. 2021. CORE: a complex event recognition engine. *arXiv preprint arXiv:2111.04635* (2021).

[9] Chainalysis Team. 2023. Case Study: How We Track Crypto Money Laundering for Off-Chain Crime. https://www.chainalysis.com/blog/japan-cryptocurrency-money-laundering-case-study/ Accessed: 2026-03-26.

[10] Sarah Chlyah, Pierre Genevès, and Nabil Layaïda. 2025. Distributed Evaluation of Graph Queries using Recursive Relational Algebra. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2351–2365.

[11] Isabel F Cruz, Alberto O Mendelzon, and Peter T Wood. 1987. A graphical query language supporting recursion. *ACM SIGMOD Record* 16, 3 (1987), 323–330.

[12] Faizan Iftikhar Janjua. 2023. Metaverse Financial Transactions Dataset. https://www.kaggle.com/datasets/faizaniftikharjanjua/metaverse-financial-transactions-dataset. Kaggle dataset. Accessed: 2026-03-26.

[13] Benjamín Farías, Wim Martens, Carlos Rojas, and Domagoj Vrgoč. 2023. Pathfinder: A unified approach for handling paths in graph query languages. *arXiv preprint arXiv:2306.02194* (2023).

[14] Amela Fejza, Pierre Genevès, Nabil Layaïda, and Sarah Chlyah. 2023. The μ-ra system for recursive path queries over graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 5041–5045.

[15] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoğlu. 2023. Kùzu graph database management system. In *The Conference on Innovative Data Systems Research*, Vol. 7. 25–35.

[16] Diego Figueira, Artur Jez, and Anthony W Lin. 2022. Data path queries over embedded graph databases. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 189–201.

[17] Diego Figueira and Leonid Libkin. 2015. Path logics for querying graphs: Combining expressiveness and efficiency. In *2015 30th Annual ACM/IEEE Symposium on Logic in Computer Science*. IEEE, 329–340.

[18] Nadime Francis, Amélie Gheerbrant, Paolo Guagliardo, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Liat Peterfreund, Alexandra Rogova, and Domagoj Vrgoc. 2023. A Researcher's Digest of GQL. In *The 26th International Conference on Database Theory, 2023*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 1–1.

[19] Roberto García, Renzo Angles, Vicente Rojas, and Sebastián Ferrada. 2025. PathDB: A system for evaluating regular path queries. *arXiv preprint arXiv:2507.01755* (2025).

[20] Amélie Gheerbrant, Leonid Libkin, Liat Peterfreund, and Alexandra Rogova. 2024. Gql and sql/pgq: Theoretical models and expressive power. *arXiv preprint arXiv:2409.01102* (2024).

[21] Amélie Gheerbrant, Leonid Libkin, and Alexandra Rogova. 2025. Dangers of List Processing in Querying Property Graphs. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–25.

[22] JIM GRAY, SURAJIT CHAUDHURI, ADAM BOSWORTH, ANDREW LAYMAN, DON REICHART, MURALI VENKATRAO, FRANK PELLOW, and HAMID PIRAHESH. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery* 1 (1997), 29–53.

[23] Daniel Gyllstrom, Eugene Wu, Hee-Jin Chae, Yanlei Diao, Patrick Stahlberg, and Gordon Anderson. 2006. SASE: Complex event processing over streams. *arXiv preprint cs/0612128* (2006).

[24] Mohamed Hassan, Tatiana Kuznetsova, Hyun Chai Jeong, Walid Aref, and Mohammad Sadoghi. 2018. Extending In-Memory Relational Database Engines with Native Graph Support. In *Proceedings of the 21st International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org. doi:10.5441/002/EDBT.2018.04

[25] Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardto, Hassan Chafio, Mihai Capotă, Narayanan Sundaram, et al. 2016. LDBC Graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms. *Proceedings of the VLDB Endowment* 9, 13 (2016), 1317–1328.

[26] Guodong Jin and Semih Salihoglu. 2022. Making RDBMSs efficient on graph workloads through predefined joins. *Proc. VLDB Endow.* 15, 5 (Jan. 2022), 1011–1023. doi:10.14778/3510397.3510400

[27] Michael Kaminski and Nissim Francez. 1994. Finite-memory automata. *Theoretical Computer Science* 134, 2 (1994), 329–363.

[28] André Koschmieder and Ulf Leser. 2012. Regular path queries on large graphs. In *Proceedings of the 24th International Conference on Scientific and Statistical Database Management* (Chania, Crete, Greece) *(SSDBM'12)*. Springer-Verlag, Berlin, Heidelberg, 177–194. doi:10.1007/978-3-642-31235-9_12

[29] Srijan Kumar, Bryan Hooi, Disha Makhija, Mohit Kumar, Christos Faloutsos, and VS Subrahmanian. 2018. Rev2: Fraudulent user prediction in rating platforms. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. ACM, 333–341.

[30] Srijan Kumar, Francesca Spezzano, VS Subrahmanian, and Christos Faloutsos. 2016. Edge weight prediction in weighted signed networks. In *Data Mining (ICDM), 2016 IEEE 16th International Conference on*. IEEE, 221–230.

[31] Srijan Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 1269–1278.

[32] Jure Leskovec. [n. d.]. Stanford Large Network Dataset Collection. https://snap.stanford.edu/data/ [Accessed: 2024].

[33] Heyang Li, Anthony Widjaja Lin, and Domagoj Vrgoč. 2025. Answering Constraint Path Queries over Graphs. *arXiv preprint arXiv:2512.01733* (2025).

[34] Qi Liang, Dian Ouyang, Fan Zhang, Jianye Yang, Xuemin Lin, and Zhihong Tian. 2024. Efficient Regular Simple Path Queries under Transitive Restricted Expressions. *Proc. VLDB Endow.* 17, 7 (March 2024), 1710–1722. doi:10.14778/3654621.3654636

[35] Leonid Libkin, Wim Martens, Filip Murlak, Liat Peterfreund, and Domagoj Vrgoč. 2025. Querying Graph Data: Where We Are and Where To Go. In *Companion of the 44th Symposium on Principles of Database Systems*. 9–26.

[36] Leonid Libkin, Wim Martens, and Domagoj Vrgoč. 2016. Querying graphs with data. *Journal of the ACM (JACM)* 63, 2 (2016), 1–53.

[37] Leonid Libkin and Domagoj Vrgoč. 2012. Regular path queries on graphs with data. In *Proceedings of the 15th International Conference on Database Theory* (Berlin, Germany) *(ICDT '12)*. Association for Computing Machinery, New York, NY, USA, 74–85. doi:10.1145/2274576.2274585

[38] Chunbin Lin, Benjamin Mandel, Yannis Papakonstantinou, and Matthias Springer. 2016. Fast in-memory SQL analytics on typed graphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 265–276. doi:10.14778/3021924.3021941

[39] Katja Losemann and Wim Martens. 2013. The complexity of regular expressions and property paths in SPARQL. *ACM Transactions on Database Systems (TODS)* 38, 4 (2013), 1–39.

[40] Wim Martens, Matthias Niewerth, Tina Popp, Carlos Rojas, Stijn Vansummeren, and Domagoj Vrgoč. 2023. Representing Paths in Graph Database Pattern Matching. *Proc. VLDB Endow.* 16, 7 (March 2023), 1790–1803. doi:10.14778/3587136.3587151

[41] József Marton, Gábor Szárnyas, and Dániel Varró. 2017. Formalising openCypher graph queries in relational algebra. In *European Conference on Advances in Databases and Information Systems*. Springer, 182–196.

[42] Memgraph. 2024. Fastest, Most Affordable Graph Database. https://memgraph.com/ [Accessed: 2024].

[43] Alberto O. Mendelzon and Peter T. Wood. 1995. Finding Regular Simple Paths in Graph Databases. *SIAM J. Comput.* 24, 6 (1995), 1235–1258. arXiv:https://doi.org/10.1137/S009753979122370X doi:10.1137/S009753979122370X

[44] Merkle Science. n.d.. Crypto Risk Scores: How They Help Prevent Financial Crime. https://www.merklescience.com/blog/crypto-risk-scores-how-they-help-prevent-financial-crime Accessed: 2026-03-26.

[45] Amine Mhedhbi, Amol Deshpande, and Semih Salihoğlu. 2024. Modern Techniques For Querying Graph-structured Databases. *Found. Trends Databases* 14, 2 (Oct. 2024), 72–185. doi:10.1561/1900000090

[46] Neo4j. 2024. Neo4j Graph Database and Analytics. https://neo4j.com/ [Accessed: 2024].

[47] Van-Quyet Nguyen, Van-Hau Nguyen, Minh-Quy Nguyen, Quyet-Thang Huynh, and Kyungbaek Kim. 2021. Efficiently Estimating Joining Cost of Subqueries in Regular Path Queries. *Electronics* 10, 9 (2021), 990.

[48] Van-Quyet Nguyen, Van-Hau Nguyen, Huy-The Vu, Minh-Quy Nguyen, Quyet-Thang Huynh, and Kyungbaek Kim. 2020. Accelerating Parallel Evaluation of Regular Path Queries on Large Graphs by Estimating Joining Cost of Subqueries.

In *The 9th International Conference on Smart Media and Applications*. 464–469.

[49] Anil Pacaci, Angela Bonifati, and M Tamer Özsu. 2020. Regular path query evaluation on streaming graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1415–1430.

[50] Mark Raasveldt and Hannes Mühleisen. 2019. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 international conference on management of data*. 1981–1984.

[51] Michael Sipser. 1996. Introduction to the Theory of Computation. *ACM Sigact News* 27, 1 (1996), 27–29.

[52] Michael Stonebraker and Andrew Pavlo. 2024. What Goes Around Comes Around... And Around... *SIGMOD Rec.* 53, 2 (July 2024), 21–37. doi:10.1145/3685980.3685984

[53] Frank Tetzel, Romans Kasperovics, and Wolfgang Lehner. 2019. Graph traversals for regular path queries. In *Proceedings of the 2nd Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*. 1–8.

[54] Domagoj Vrgoč. 2015. Using variable automata for querying data graphs. *Inform. Process. Lett.* 115, 3 (2015), 425–430.

[55] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. 2023. MillenniumDB: An Open-Source Graph Database System. *Data Intelligence* 5, 3 (08 2023), 560–610. arXiv:https://direct.mit.edu/dint/article-pdf/5/3/560/2158194/dint_a_00229.pdf doi:10.1162/dint_a_00229

[56] Sarisht Wadhwa, Anagh Prasad, Sayan Ranu, Amitabha Bagchi, and Srikanta Bedathur. 2019. Efficiently Answering Regular Simple Path Queries on Large Labeled Networks. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1463–1480. doi:10.1145/3299869.3319882

[57] Daniel ten Wolde, Gábor Szárnyas, and Peter Boncz. 2023. Duckpgq: Bringing sql/pgq´ to duckdb. *Proceedings of the VLDB Endowment* 16, 12 (2023), 4034–4037.

[58] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2013. Evaluation of SPARQL Property Paths via Recursive SQL. *AMW* 1087 (2013).

[59] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2015. WAVEGUIDE: Evaluating SPARQL Property Path Queries.. In *EDBT*, Vol. 2015. 525–528.

[60] Nikolay Yakovets, Parke Godfrey, and Jarek Gryz. 2016. Query planning for evaluating SPARQL property paths. In *Proceedings of the 2016 International Conference on Management of Data*. 1875–1889.

[61] Guangyi Yang, Xiaoxing Liu, and Beixin Li. 2023. Anti-money laundering supervision by intelligent algorithm. *Computers & Security* 132 (2023), 103344.

Author Feedback

We thank all reviewers for their helpful feedback. Below we provide clarification and outline our revision plan. We would like to emphasize the following: In hindsight the idea of ReCAP may indeed seem intuitive and an expert could hand-craft recursive SQL queries that maintain state in the recursive part to filter on property constraints. However, (a) no SOTA system actually does this for property constraints and (b) our main contribution is the abstraction plus a compiler to produce an optimized recursive SQL query plus generality in supporting all regex and property constraints, including complex scenarios like in query Q_B where regex and property constraints interact.

R2.O1, artifacts feedback: Clarification regarding *'"implemented in DuckDB" is highly misleading. The provided code artifacts provided by the authors do not contain any extensions to DuckDB that would justify this claim'*: We indeed did not extend DuckDB and we do not claim to extend it. We use DuckDB as one of the possible environments to implement ReCAP, hence it is "implemented in DuckDB"—like "this algorithm is implemented in Java" means that Java was used to implement it, not that Java was extended.

We disagree with the *'hard-coded prototype'* characterization. Our framework is fully general as we prove in Section 5.3 and as can be seen in Listing 3. Of course, when applying it to specific queries, it will produce query-specific code for the selective aggregate and the regex. Our artifacts contain two distinct components: The first are specific SQL implementations of our benchmark queries as produced by our framework. But the second is the ReCAP *workbench*, an interactive browser-based tool where the user defines a new ReCAP instance. Due to time constraints, this code comes with a single dataset (which is trivial to extend to let the user pick others) and has a hard-coded default example to illustrate its functionality. But when entering a new regex, the skeleton code gets updated automatically. To avoid confusion we will extend the workbench as outlined in the revision plan so that it provides full flexibility to select data, supports all regex, and includes the code to automatically generate the final SQL query.

R2.O2: To claim that our work is *'textbook knowledge'* conflates "what" with "how." Our experiments show that SOTA graph and relational DBMS do not know how to effectively push filters on properties deep into the query plan. Formalizing negative stability, the selective aggregate, general constructions (Section 5.1), and demonstrating 400,000× speedups do not follow from any textbook solution we are aware of.

R2.O3: Our approach provides a lot more automation than the reviewer gives us credit for, and we think the redesigned workbench will address this confusion. Note that automatic identification of negatively stable constraints for a given query is a hard open problem that is part of our ongoing research. To draw a parallel, proposing object-oriented programming (OOP) abstractions like classes is a useful contribution, even if one cannot auto-generate class definitions from a problem spec.

R3.O1: As proposed by the reviewer, we will add an experiment with label-regex-only queries of varying complexity and selectivity, plus versions with additional property constraints, to isolate the difference made by early filtering for property constraints.

R3.O2: Due to the combinatorial nature of path queries, even small inputs can produce huge (intermediate) results. And trail queries are known to be hard. This affects all approaches, but especially the SOTA competitors. Hence we focused on graphs were the competitors can still get at least to path length 3 or 4.

R3.O3: ReCAP can indeed return paths and we will explain that more clearly. Note that all benchmark queries use trail semantics, hence keep track of all edge IDs. This means that in all experiments, ReCAP is indeed managing full paths. (This is different from some previous work that considers queries that only return the reached path endpoints, which would allow aggressive pruning.) In short, despite ultimately returning a path count, all queries in the benchmark force ReCAP to explicitly manage all distinct (intermediate) paths that are viable. So, we fully stress-test our approach. We can report the corresponding memory use as well.

R3 (Regex/References): Regex limitation is a library issue. In the revision, we will implement the full Thompson's construction. We will also update the references as suggested.

R4.O1: Thank you for the thoughtful points made, which helped us identify ways to better present our contributions.

(a) Clarifying what Listing 1 demonstrates. Our statement that "computing a path aggregate like the maximum across all edges indeed requires knowing all edges" was not meant as a claim about query semantics. We apologize for the ambiguity: the sentence refers specifically to the *encoding in Listing 1*, where `list_max(...)-list_min(...)` is opaque to the optimizer, resulting in this constraint to be applied at the very end. We did not intend to claim that an experienced user is unable to write code to incrementally maintain a max, min, or edge-ID set inside the recursive CTE. In fact, that is exactly how we began this work: by hand-coding such queries before inventing ReCAP. The difficulty we point out here is about the *optimizer*, not the user.

(b) Origin and purpose of ReCAP. ReCAP arose from observing patterns across our handcrafted incremental encodings: Each maintained per-path state, exposed a *viability* predicate that depended on the path's state, and updated the state along transitions in the NFA. The selective aggregate generalizes these patterns. ReCAP is not an alternative to "pushing constraints into RECURSIVE" — it systematically turns that very approach into a reusable abstraction with framework-level benefits. In short, we agree that an expert can manually produce incremental code for a negatively stable constraint. ReCAP captures what those manual solutions have in common and reaps the benefits.

(c) Benefits of the abstraction beyond a 'veneer of declarativeness':

(1) Like object-oriented design for coding and user-defined aggregates (UDA) in DBMS, it provides a clear structure for expressing filtering opportunities. For an aggregate like variance one could manually inline code fragments in an SQL query to initialize, update and finalize count, sum, and sum of squares. Instead, UDAs encapsulate this in a few functions, which motivated the concept of a selective aggregate in ReCAP.

(2) With ReCAP, the user determines correctness of small independently testable functions plus an invariant: a path is an answer iff it reaches an accepting NFA state and `is_viable_d_final` returns true. The correctness burden

remains with the user, but it is *localized* to small, independently verifiable functions rather than spread across an entire recursive SQL query.

(3) Decisions like dictionary representation (JSON vs flat columns vs structs) and DBMS-vendor-specific encoding live in ReCAP's compilation layer rather than in hand-crafted queries. Improvements to the ReCAP compiler (e.g., dictionary flattening, function inlining, or a new target system) benefit every ReCAP specification automatically. A hand-crafted SQL query bakes these choices in and must be individually customized per query and per system. Like UDAs, ReCAP also enables a reusable library of selective aggregates — particularly for factorized cases, where the NFA-independent skeleton can also be used to scaffold non-factorized ones.

(d) ReCAP is load-bearing for a broader range of execution strategies. We will clarify that ReCAP is compatible with more than the fixed execution order of current implementations of recursive SQL queries. (See also response to R4.O2.) Consider Q_B (Example 5): A hand-crafted implementation could dispense with the NFA entirely. Because the regex `Domestic`$^+$ `·Foreign` has the property that `last_label` uniquely determines the NFA state, the user might decide to incrementally maintain `last_label`, `last_time`, and `edge_ids`. This works for Q_B, but it forecloses wavefront-style optimization: there is no NFA in the hand-crafted version to split and no transition to reverse. ReCAP preserves the NFA and remains compatible with these strategies. While the hand-crafted shortcut may win on current DBMS, it may lose on the next generation that may support more advanced plans for recursive SQL queries.

R4.O2: This is a great point, and we are happy to add a discussion along the following lines: Indeed, ReCAP as discussed relies on the current evaluation strategy for recursive queries. However, negative stability is a property of the constraint, not of the query plan. And our *intentional design decision* to formalize ReCAP as an NFA with functions called along state transitions allows ReCAP to benefit from the richer query-plan space of approaches like wavefronts that can split a regex, prioritize less common labels, and even reverse the matching direction [28, 58–60]. That work focused on label regex. Because the selective aggregate is parameterized by NFA transitions, ReCAP is compatible with these approaches as we can analogously split the regex into pieces and invert edge direction. Instead of several "mini" NFAs, we would obtain the corresponding "mini" ReCAPs. Of course, for inverted edge direction, one would have to provide the "reverse" versions of `update_d` and `is_viable_d`. E.g., for monotonicity, the reverse `is_viable_d` flips "<" to ">" etc.

Proposed experiment. We propose to add a benchmark that directly tests R4's concern: (i) Handcrafted recursive CTEs with constraints inlined in the recursive part, (ii) ReCAP-Optimized under naive forward BFS (current paper), (iii) ReCAP-Optimized under a WAVEGUIDE-style planner that splits the regex at the most selective label and may reverse direction.

We expect (i) and (ii) to be comparable on the queries as written — confirming that ReCAP's abstraction cost is negligible. We expect (iii) to deliver further speedup on queries with selective rare labels, with the gain only accessible because ReCAP preserved the NFA. This isolates the abstraction's contribution from the early-filtering contribution and directly demonstrates that the structure ReCAP exposes is load-bearing rather than cosmetic.

R4.O3 (SQL listings): We explain s in the text and will improve the listing to avoid confusion.

R4.O3 (Related work): This was caused by last-minute cutting. We will make a thorough pass over all references and correct any issues. Note that the main message remains that none of these approaches addressed the property constraints targeted by ReCAP.

R4 (Minor): Will replace "SOA" with "SOTA."

R5.O1: Our benchmark is actually inspired by the LDBC FinBench. TCR1, TCR2, TCR5 require strictly ascending timestamps on multi-hop transfer paths—the negatively stable monotonicity in Q_A, Q_B. TCR8 enforces per-step amount thresholds (negatively stable aggregate constraints). Crucially, FinBench sidesteps the scalability issue by capping paths at a small constant ("at most 3 steps"), and even then the reference Cypher implementations resort exactly to the opaque list processing our paper identifies as the core problem: TCR5's reference query collects timestamps into a list and checks monotonocity via `reduce` fold over a list—a pattern that forces evaluation over complete paths. ReCAP enables *lifting this restriction* without sacrificing early filtering. We will adapt FinBench queries to the variable-length setting and include end-to-end results.

R5.O2, R5.O3: Will fix.

R5.M1–M9: We will improve the presentation as suggested. For **M3** to support conditions on vertex properties, one includes the vertex table in the join. Since this is straightforward, we omitted it. For **M9**, please note: Figure 9 = ReCAP-Optimized. Q_2 speedup over ReCAP-Standard is from eliminating JSON/UDF overhead; and relative to the competitors speedup is from incremental color piggybacking. Will clarify.

Revision Plan

In addition to clarifications and presentation improvements (R3.O1, R4.O2, R4.O3, R5.O2-O3, M1-M9, R5.O2), we propose:

New experiments:

(1) Quantify how much of the ReCAP improvement is due to the automata-driven exploration vs early pruning of data constraints (R3.O1).
(2) Demonstrate benefits of ReCAP enabling advanced optimizations (see response to R4.O2).
(3) FinBench queries (TCR1/TCR5) lifted to variable-length paths with end-to-end results (R5.O1).
(4) Datagen-7.7 experiments for additional queries (R3.O2).
(5) Report memory usage (R3.O3).

Framework and artifact improvements:

(6) Redesigned workbench: user selects data and start vertices → user specifies regex → system automatically generates NFA and skeleton code for selective aggregate → user modifies skeleton code → system automatically generates optimized SQL query with dictionary flattening and inlining (R2.O1, R2.O3, R4.O1).
(7) Replace all Python code with SQL code (R2.O1).
(8) Add library of selective aggregates for creation of skeleton code (R4.O1, R2.O1).
(9) Add full Thompson's construction to support all regex (R3).